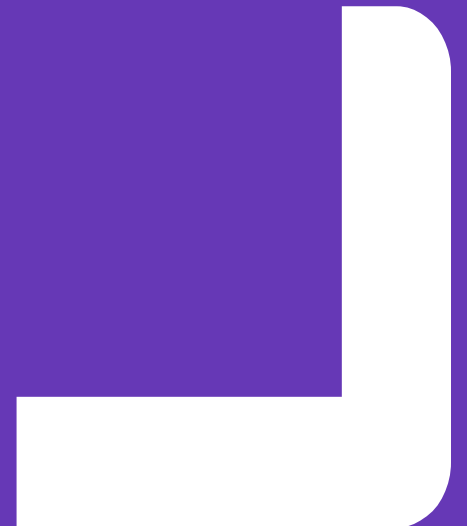


SOFTWARE ENGINEERING



Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de

┌ ENTWURFSMUSTER (DESIGN PATTERNS)



Themen in dieser Vorlesung



Entwurfsmuster (Design Patterns)

- standardisierte Lösungen für gängige Entwurfsprobleme auf Ebene des Feinentwurfs von Komponenten
- **unabhängig** von bestimmter Sprache formuliert, meist für objektorientierte Konzepte

Charakterisierung von Gamma, Helm, Johnson und Vlissides:

Design patterns ... are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context.

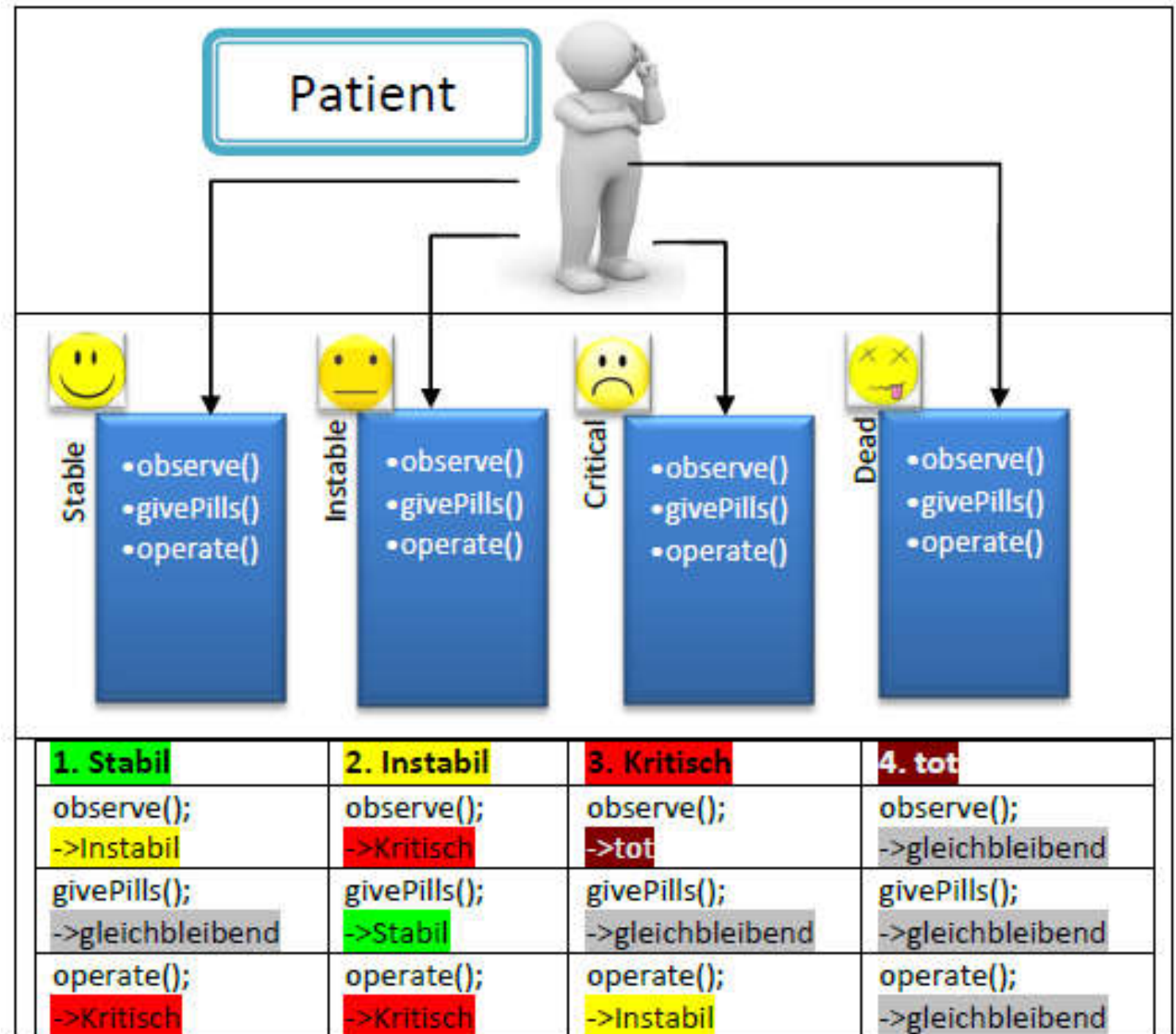
A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object-oriented design.

Entwurfsmuster kurz gefaßt

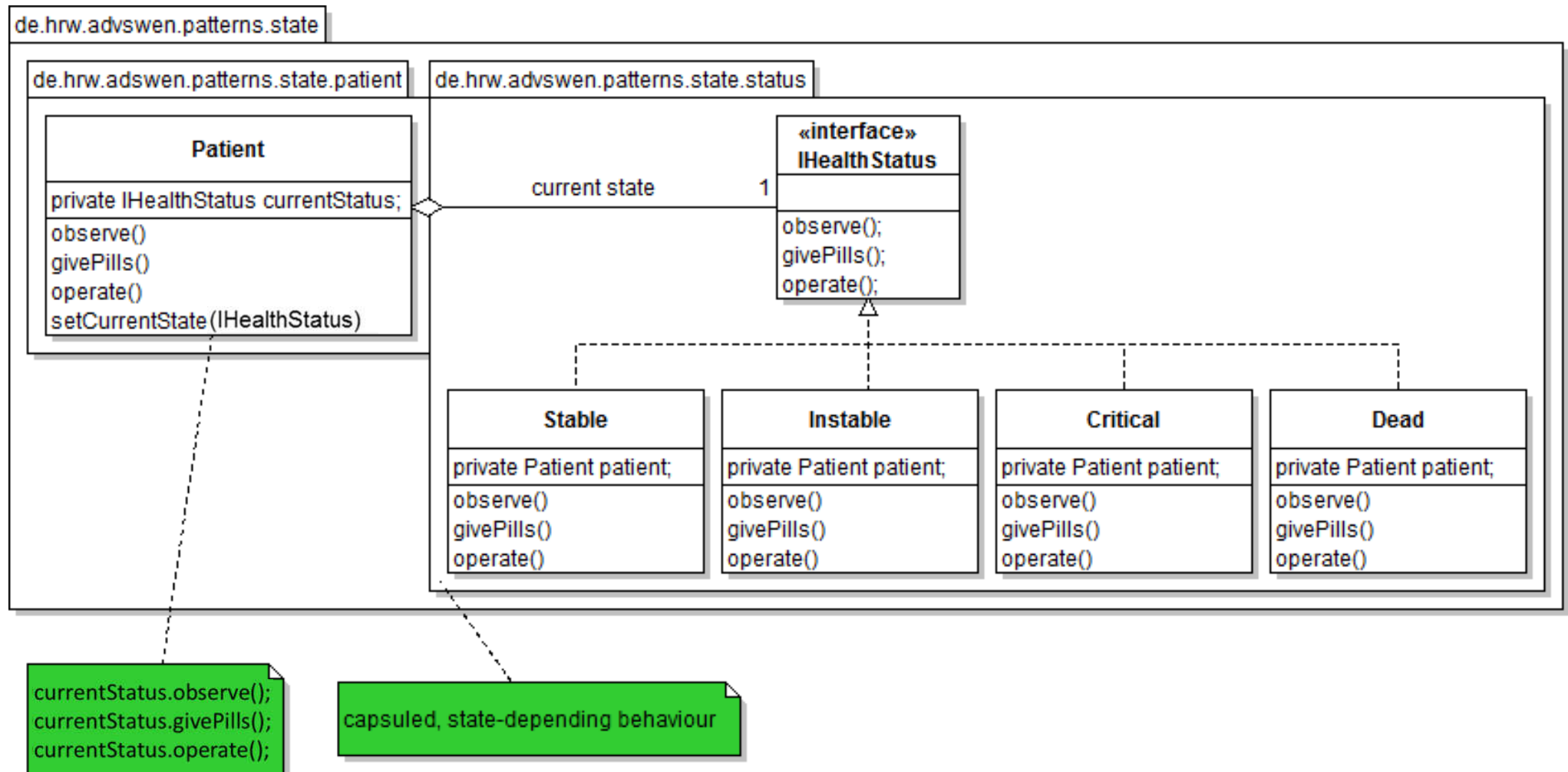
Also: Vordefinierte objektorientierte Entwürfe

- für spezifische Probleme
- mit einheitlicher Beschreibung,
- wiederverwendbar aber abstrakt beschrieben,
- müssen deshalb bei Einsatz in jeweiliger Situation konkretisiert werden: Man muß wissen, **welches Problem es löst** und wie man das Entwurfsmuster im Kontext einer konkreten Architektur **ausprägen** kann

State- Pattern



State-Pattern

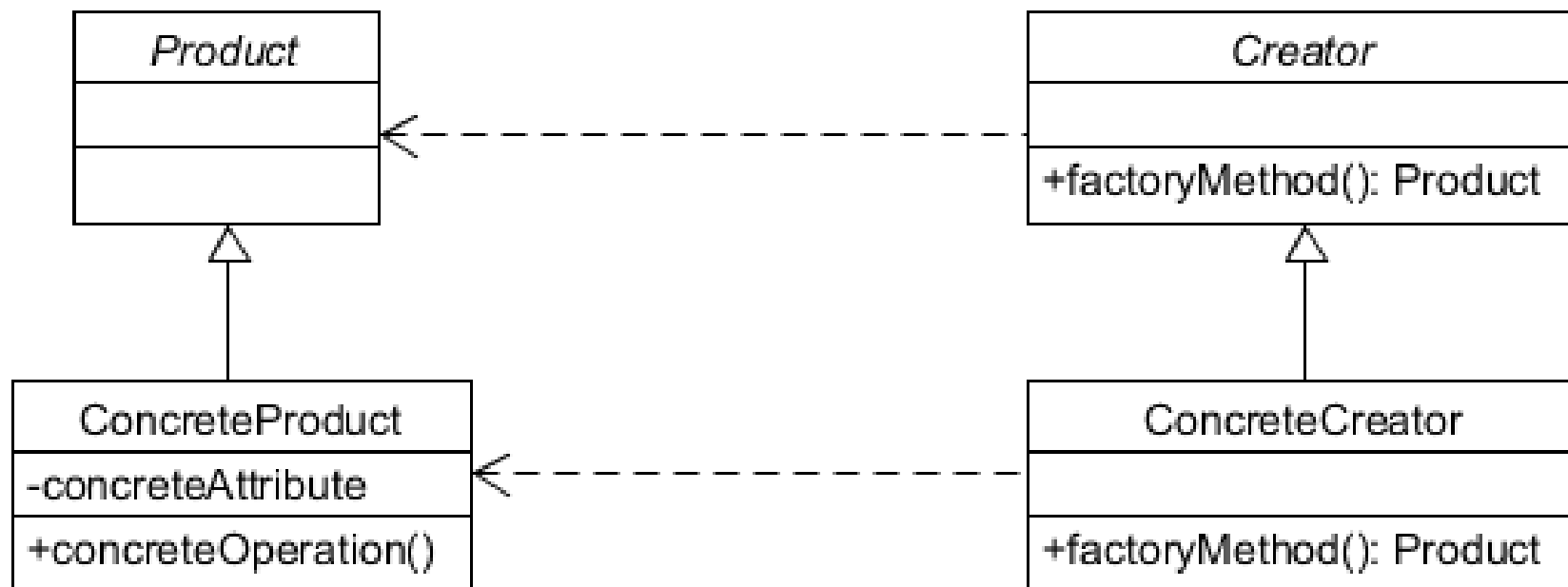


Entwurfsmuster: State

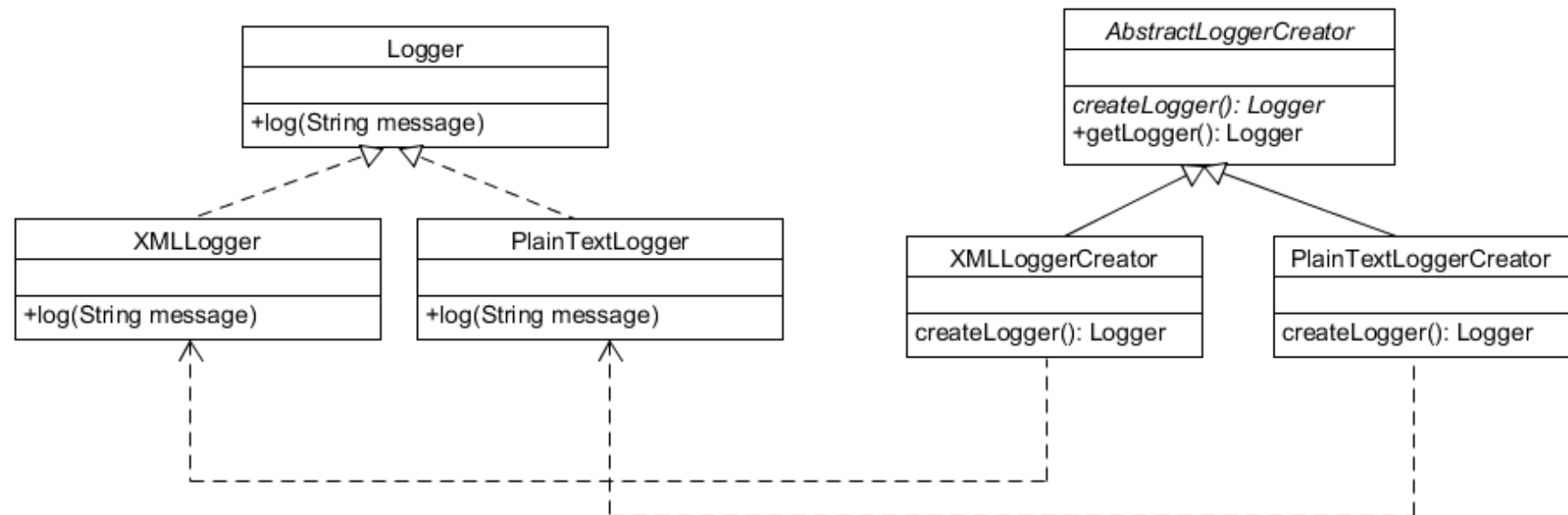
Absicht	Einem Objekt erlauben, sein Verhalten zu ändern wenn sein interner Zustand sich verändert. Im Extremfall kann es wirken, als ob das Objekt seinen Typ geändert hätte.
Andere Namen	Objects for States
Muster- klassifizierung	Objektverhalten
Motivation	Manchmal können Objekte in einem von mehreren möglichen Zuständen sein, und das über seine Methoden abrufbare Verhalten des Objektes soll sich je nach Zustand des Objektes unterscheiden. Jede Methode muss alle Zustände berücksichtigen, die Methoden werden dadurch komplex. Zusätzlich ist das Wissen über die Zustände ist dann über alle zugehörigen Methoden verstreut. State bietet einen Weg, das Wissen und Verhalten zu Zuständen in einzelnen Klassen zu kapseln.
Anwendung	• Wenn das Verhalten eines Objektes von seinem Zustand abhängt und zur Laufzeit in Abhängigkeit von diesem Zustand wechseln muss • Wenn Methoden große if-else- oder switch-Anweisungsblöcke enthalten, die vom Zustand eines Objektes abhängen

Factory Method

Entkoppelung von konkreten Typen: Erzeugung von konkreten Objekten in austauschbarer Fabrik



Factory Method: Beispiel



Factory Method: Beispiel

```
public interface Logger {  
    public void log(String message);  
}
```

Produkt
(ohne PlainTextLogger)

```
public class XMLLogger implements Logger {  
  
    * (non-Javadoc)   
    @Override  
    public void log(String message) {  
        System.err.println("<message>" + message + "</message>");  
    }  
  
}
```

Factory Method: Beispiel

```
public abstract class AbstractLoggerCreator {  
    public abstract Logger createLogger();  
  
    public Logger getLogger() {  
        Logger logger = createLogger();  
  
        // hier könnte man den Logger noch allgemein  
        // initialisieren...  
  
        return logger;  
    }  
}
```

Erzeugung
(ohne PlainTextLoggerCreator)

```
public class XMLLoggerCreator extends AbstractLoggerCreator {  
    * (non-Javadoc)   
    @Override  
    public Logger createLogger() {  
        XMLLogger logger = new XMLLogger();  
  
        // hier könnte man speziell den XMLLogger noch  
        // initialisieren...  
  
        return logger;  
    }  
}
```

Factory Method: Beispiel

Verwendung

```
10 public class FactoryMethodClient {
11
12     private AbstractLoggerCreator logCreator;
13
14     public FactoryMethodClient(AbstractLoggerCreator logCreator) {
15         this.logCreator = logCreator;
16     }
17
18     public void produceSomeLogging() {
19         Logger logger = logCreator.getLogger();
20         logger.log("some text");
21     }
22
23     /**
24      * @param args
25      */
26     public static void main(String[] args) {
27         FactoryMethodClient client = new FactoryMethodClient(new XMLLoggerCreator());
28         client.produceSomeLogging();
29     }
30
31 }
```

Entwurfsmuster: Factory Method

Absicht	Eine Schnittstelle zur Objekterzeugung bereitstellen, aber Unterklassen über Typen entscheiden lassen.
Andere Namen	Virtual Constructor
Muster-klassifizierung	Objekterzeugung
Motivation	Manchmal möchte man erst möglich spät festlegen und oder zur Laufzeit dynamisch festlegen, welche konkreten Objekte zu einem abstrakten Typ tatsächlich erzeugt werden sollen.
Anwendung	• Wenn eine Klasse nicht vorhersehen kann, von welcher Klasse sie Objekte erzeugen soll • Wenn eine Klasse eine Unterklasse entscheiden lassen möchte, welche Objekte erzeugt werden sollen • Wenn das Wissen über die Erzeugung der Unterklassen an einem Ort gekapselt werden soll

Decorator: Mögliches Szenario

Hotel mit

- Basisleistung
- Mediumleistung
- Premiumleistung

Zusätzlich buchbar:

- Essen auf dem Zimmer
- Fernseher

Decorator: Beispiel

abstrakte Komponente und konkrete Komponente

```
package de.hrw.swen.patterns.decorator;

public abstract class Serviceleistung {
    protected String name;
    protected double preis;

    public abstract String getName();

    public abstract double preis();
}
```

```
public class MediumLeistung extends Serviceleistung {

    public MediumLeistung() {
        preis = 350.0;
    }

    @Override
    public String getName() {
        return "MediumLeistung";
    }

    @Override
    public double preis() {
        return preis;
    }
}
```


Decorator: Beispiel

```
public abstract class ZusatzleistungDecorator extends Serviceleistung {  
  
    protected Serviceleistung zuErweiterndeLeistung;  
  
    public ZusatzleistungDecorator(Serviceleistung zuErweiterndeLeistung) {  
        this.zuErweiterndeLeistung = zuErweiterndeLeistung;  
    }  
  
    /* (non-Javadoc)␣  
    @Override  
    public String getName() {  
        return zuErweiterndeLeistung.getName();  
    }  
  
    /* (non-Javadoc)␣  
    @Override  
    public double preis() {  
        return zuErweiterndeLeistung.preis();  
    }  
}
```

abstrakter Dekorierer und
konkreter Dekorierer

```
public class EssenAufZimmer extends ZusatzleistungDecorator {  
  
    * @param zuErweiterndeLeistung␣  
    public EssenAufZimmer(Serviceleistung zuErweiterndeLeistung) {  
        super(zuErweiterndeLeistung);  
        preis = 25.0;  
    }  
  
    @Override  
    public String getName() {  
        return super.getName() + " mit Essen auf dem Zimmer";  
    }  
  
    @Override  
    public double preis() {  
        return super.preis() + preis;  
    }  
}
```

Decorator: Beispiel

Verwendung

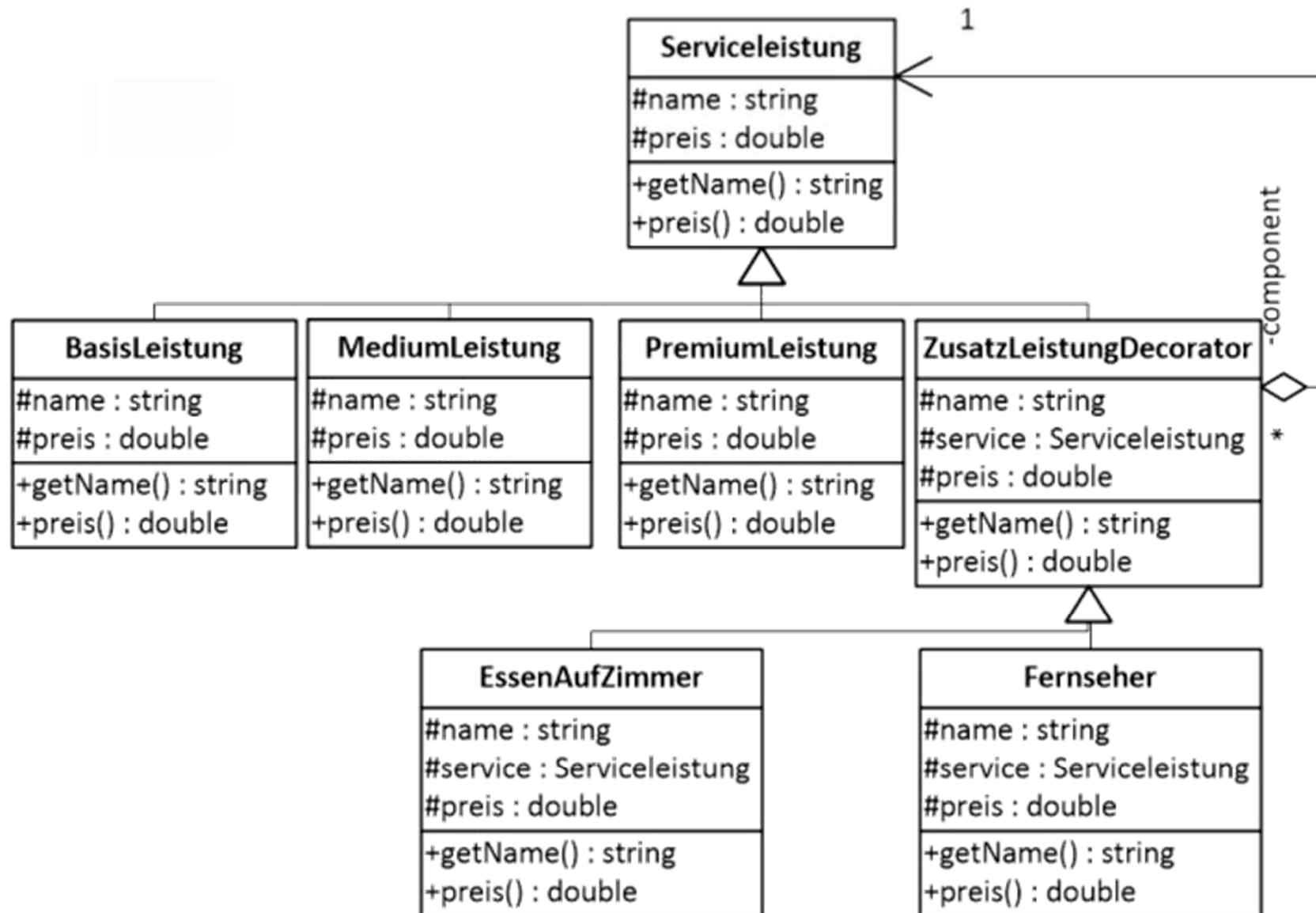
```
public class DecoratorTest {  
  
    * @param args[]  
    public static void main(String[] args) {  
        Serviceleistung leistung = new EssenAufZimmer(new Fernseher(new MediumLeistung()));  
        System.out.println(leistung.getName());  
        System.out.println(leistung.preis());  
    }  
}
```

Problems | @ Javadoc | Declaration | Console ✕

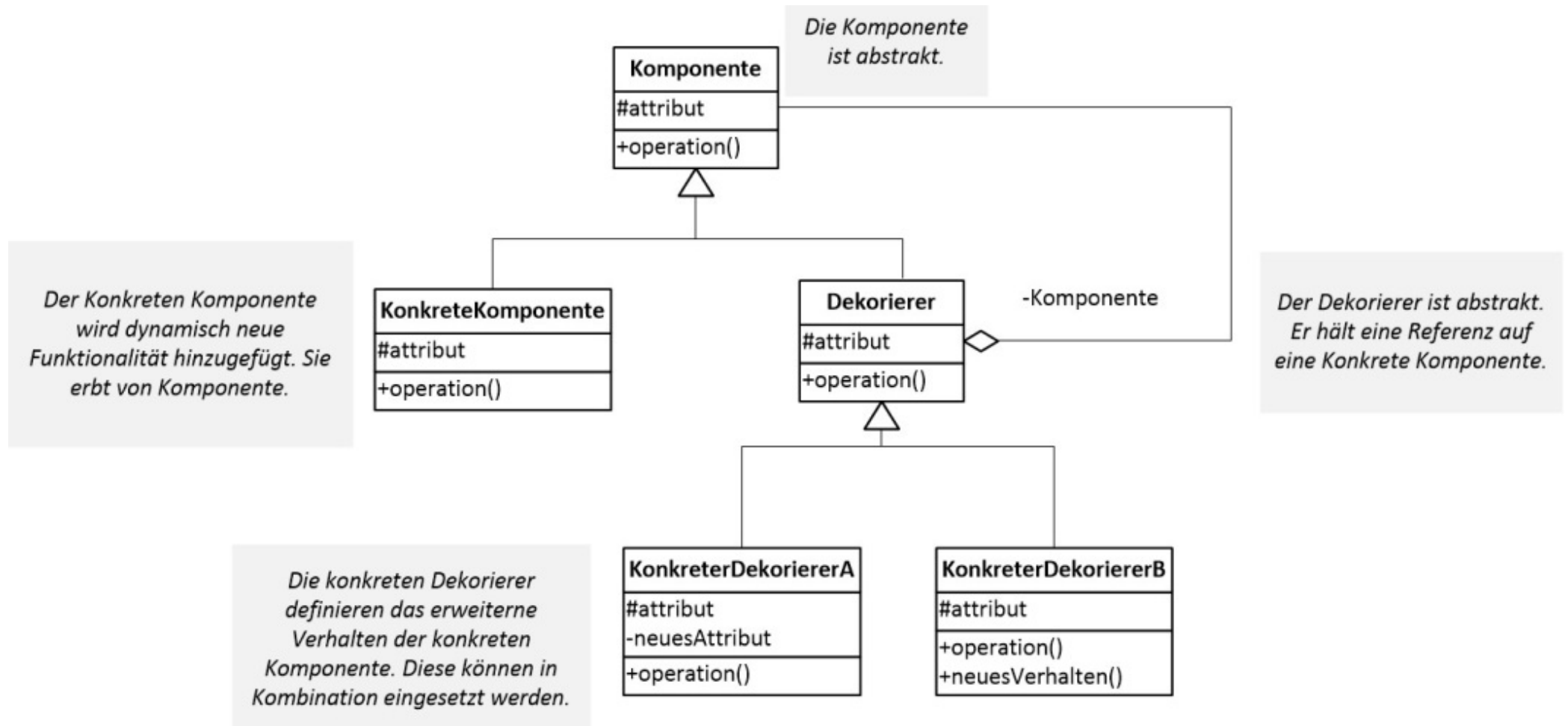
<terminated> DecoratorTest [Java Application] C:\Program Files\Java\jdk1.8.0_25\bin\javaw.exe

MediumLeistung mit Fernseher mit Essen auf dem Zimmer
385.0

Decorator: Beispiel



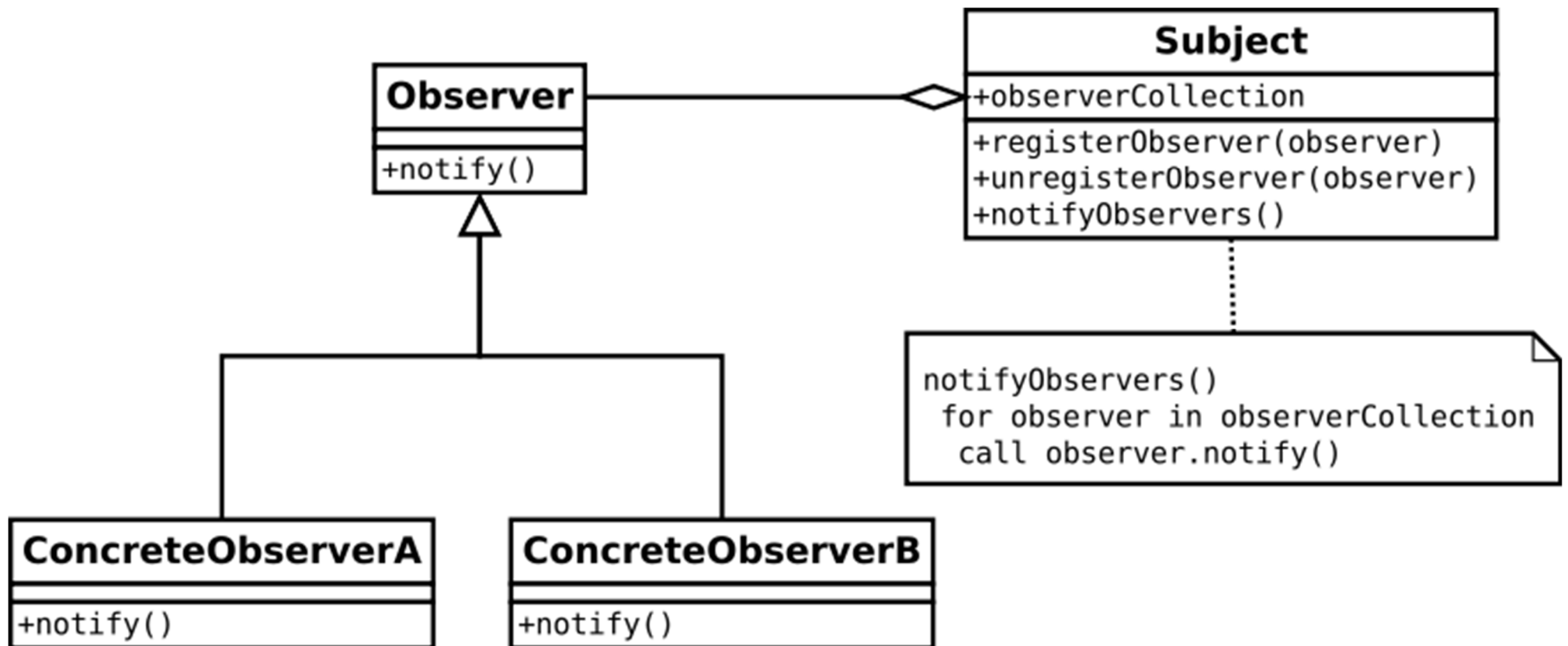
Decorator abstrakt



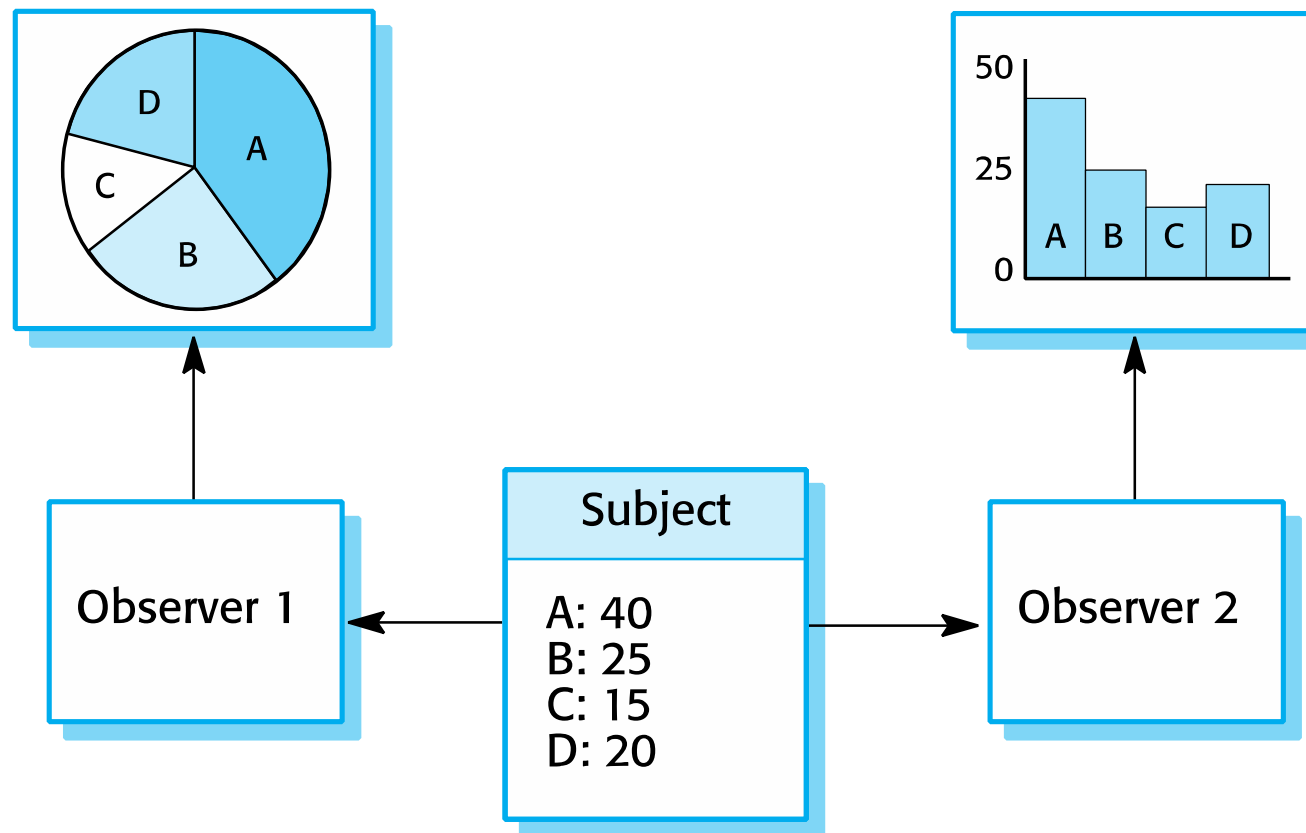
Entwurfsmuster: Decorator

Absicht	Ein Objekt dynamisch mit zusätzlichen Funktionalitäten versehen. Eine flexible Alternative zum Vererben.
Andere Namen	Wrapper
Muster-klassifizierung	Objektstruktur
Motivation	Manchmal möchte man nur einzelne Objekte und nicht ganze Klassen mit zusätzlicher Funktionalität ausstatten. Wenn wir Vererbung benutzen ist das unflexibel: Wollen wir beispielsweise ein Fenster mit einem Rahmen versehen und lösen dies mit Vererbung, haben wir immer entweder ein Fenster mit oder ohne Rahmen und nicht ein Fenster, das je nach Situation einen Rahmen haben kann oder nicht. Bei einer großen Zahl von Zusatzfunktionalitäten und wenn diese miteinander kombiniert werden könnten, ist Vererbung auch aufgrund der großen Zahl möglicher Unterklassen keine Lösung.
Anwendung	• Wenn einzelne Objekte und nicht Klassen dynamisch und transparent (ohne andere Objekte zu beeinflussen) neue Funktionalität erhalten sollen • Wenn Funktionalitäten auch wieder entfernt werden sollen • Wenn Vererbung keine Lösung ist (viele, miteinander kombinierbare Zusatzfunktionalitäten)

Observer-Pattern



Observer-Pattern: Verwendung



Realisierung mehrfacher Datenanzeige zum gleichen Datensubjekt

Entwurfsmuster: Observer

Absicht	Eine 1:n-Abhängigkeit zwischen Objekten so definieren dass wenn das eine Objekt sich ändert, die n anderen Objekte informiert werden
Andere Namen	Publish-subscribe
Muster-klassifizierung	Objektverhalten
Motivation	Zerlegt man ein System in eine Menge miteinander kooperierender Klassen, muss man die Konsistenz der Daten in den entsprechenden Objekten sicherstellen, auch wenn einzelne Objekte sich ändern. Dabei will man aber die Objekte deshalb nicht eng miteinander koppeln. Observer zeigt, wie eine solche Abhängigkeit bei relativ loser Koppelung zwischen einer großen Zahl von Objekten definiert werden kann.
Anwendung	• Wenn eine Abstraktion zwei Aspekte hat, von denen der eine vom anderen abhängt. Mit Observer kann man beide Aspekte in verschiedene Klassen kapseln, die unabhängig voneinander variieren können. • Wenn die Veränderung eines Objektes die Veränderung anderer Objekte erfordert, insbesondere wenn man nicht weiß wie viele Objekte betroffen sein könnten • Wenn ein Objekt andere Objekte informieren können sollte ohne Annahmen über diese anderen Objekte zu treffen

Was leisten Entwurfsmuster?

- Externalisiertes Expertenwissen: Sprachunabhängige Wiederverwendung erfolgreicher Lösungen
- Adressieren oft einzelne Aspekte und erlauben Varianz oder Evolution für diese
- Einheitliches Vokabular
 - Unterstützung von Abstraktion der (Entwurfs-)Probleme
 - Grundbausteine -> Erhöhung der Entwurfsgeschwindigkeit
 - Getrennte Diskussion über Abstraktionen und deren Implementierungen
 - Repräsentation einer „Entwurfskultur“
- Verbesserte Dokumentation
- Verbessertes (Gesamt-)Verständnis

Kritische Betrachtung von Entwurfsmustern

- Es ist genauso wichtig zu wissen, wann ein bestimmtes Entwurfsmuster *nicht* einzusetzen ist
- Jedes Entwurfsmuster stellt letztlich einen Kompromiss dar
- Viele Entwurfsmuster führen zu komplexeren Entwürfen: Mehr Klassen, mehr Indirektionen, mehr Nachrichten werden benötigt, erzeugt oder ausgetauscht
- Benutzen Sie Entwurfsmuster sorgsam und mit gutem Grund!

Zusammenfassung

- Die Befassung mit Entwurfsmustern lohnt, weil man schnell viel über Entwurf lernen kann
- Entwurfsmuster erlauben Wissensweitergabe zu Entwurf, beschleunigen Kommunikation und Entwurf
- Man muss genau abwägen, ob ein Entwurfsmuster im jeweiligen Fall nützt oder schadet

